

```
In [31]: # this line makes figures interactive in Jupyter notebooks
%matplotlib inline
from matplotlib import pyplot as plt

import numpy as np
from scipy.optimize import root_scalar
from scipy import constants
import cantera as ct

from pint import UnitRegistry
ureg = UnitRegistry()
Q_ = ureg.Quantity

# for convenience:
def to_si(quant):
    '''Converts a Pint Quantity to magnitude at base SI units.
    ...

    return quant.to_base_units().magnitude
```

```
In [ ]:
```

```
In [32]: def vertical_launch(t, y, spec_impulse, mass_prop, time_burn, gamma):
    '''Evaluates system of time derivatives for velocity, altitude, and mass.
    ...

    radius_earth = 6378.388 * 1e3
    gravity_ref = 9.80665
    density_ref = 1.225
    pressure_ref = 101325
    height_den = 10400
    height_pres = 8400

    v = y[0]
    h = y[1]
    m = y[2]

    gravity = gravity_ref * (radius_earth / (radius_earth + h))**2
    density = density_ref * np.exp(-h / height_den)
    pressure = pressure_ref * np.exp(-h / height_pres)

    mach = v / np.sqrt(gamma * pressure / density)
    #area = np.pi * diameter**2 / 4
    #drag = 0.5 * density * v**2 * area * coef_drag(mach)

    dmdt = -mass_prop / time_burn
    dvdt = (-spec_impulse * gravity_ref / m) * dmdt - gravity
    dhdt = v

    return [dvdt, dhdt, dmdt]
```

```
In [33]: def get_area_ratio(pressure_ratio, gamma):
    '''Calculates area ratio based on specific heat ratio and pressure ratio.
    pressure ratio: chamber / exit
    area ratio: exit / throat
```

```

'''
return (
    np.power(2 / (gamma + 1), 1/(gamma-1)) *
    np.power(pressure_ratio, 1 / gamma) *
    np.sqrt((gamma - 1) / (gamma + 1) /
            (1 - np.power(pressure_ratio, (1 - gamma)/gamma))
    )

def root_area_ratio(pressure_ratio, area_ratio, gamma):
    ''' Returns zero for a given area ratio, pressure ratio, and gamma.
    pressure ratio: chamber / exit
    area ratio: exit / throat
    '''
    return area_ratio - get_area_ratio(pressure_ratio, gamma)

def get_thrust_coeff(pressure_ratio, gamma):
    '''Calculates thrust coefficient for optimum expansion.
    pressure ratio: chamber / exit
    area ratio: exit / throat
    '''
    return np.sqrt(
        2 * np.power(gamma, 2) / (gamma - 1) *
        np.power(2 / (gamma + 1), (gamma + 1)/(gamma - 1)) *
        (1 - np.power(1.0 / pressure_ratio, (gamma - 1)/gamma))
    )

def get_cstar(Tc, MW, gamma):
    '''Calculates cstar using chamber properties.'''
    return (
        np.sqrt(Ru * Tc / (gamma * MW)) *
        (2 / (gamma + 1))**(-0.5*(gamma + 1)/(gamma - 1))
    )

```

```

In [34]: # Using this formulation, derive expressions for the port radius, fuel flow, and O/
def get_port_radius(a, n, mdoto, t, Ri):

    return (a * (2*n + 1) * (mdoto / np.pi)**n * t + Ri**(2*n+1))**(1/(2*n+1))

#Fuel Flow Expression as a function of time.
#All other values in the input will be given.
def get_fuel_flow(a, n, mdoto, t, Ri, fuel_density, L):
    R_t = get_port_radius(a, n, mdoto, t, Ri)
    return 2 * np.pi * fuel_density * L * a * (mdoto / np.pi)**n * R_t**(1-2*n)

#O/F ratio expression as a function of time.
#All other values in the input will be given.
def get_O_F_ratio(a, n, mdoto, t, Ri, fuel_density, L):
    mdotf = get_fuel_flow(a, n, mdoto, t, Ri, fuel_density, L)

    return (mdoto / mdotf)

```

```

In [35]: delta_velocity = Q_(2300, 'm/s')
        mpl = Q_(100000, 'kg')

```

```
propellant_mass_fraction = 0.9
burn_time = Q_(80, 's')
Df = Q_(3, 'm')
Pc_initial = Q_(1000, 'psi')
g0 = Q_(9.81, 'm/s^2')
```

```
In [36]: mixture_ratios = np.arange(1, 3.05, 0.1) #Mixture ratios tested
with open('HTPB_LOX_CEA.txt', 'r') as f:
    lines = f.readlines()

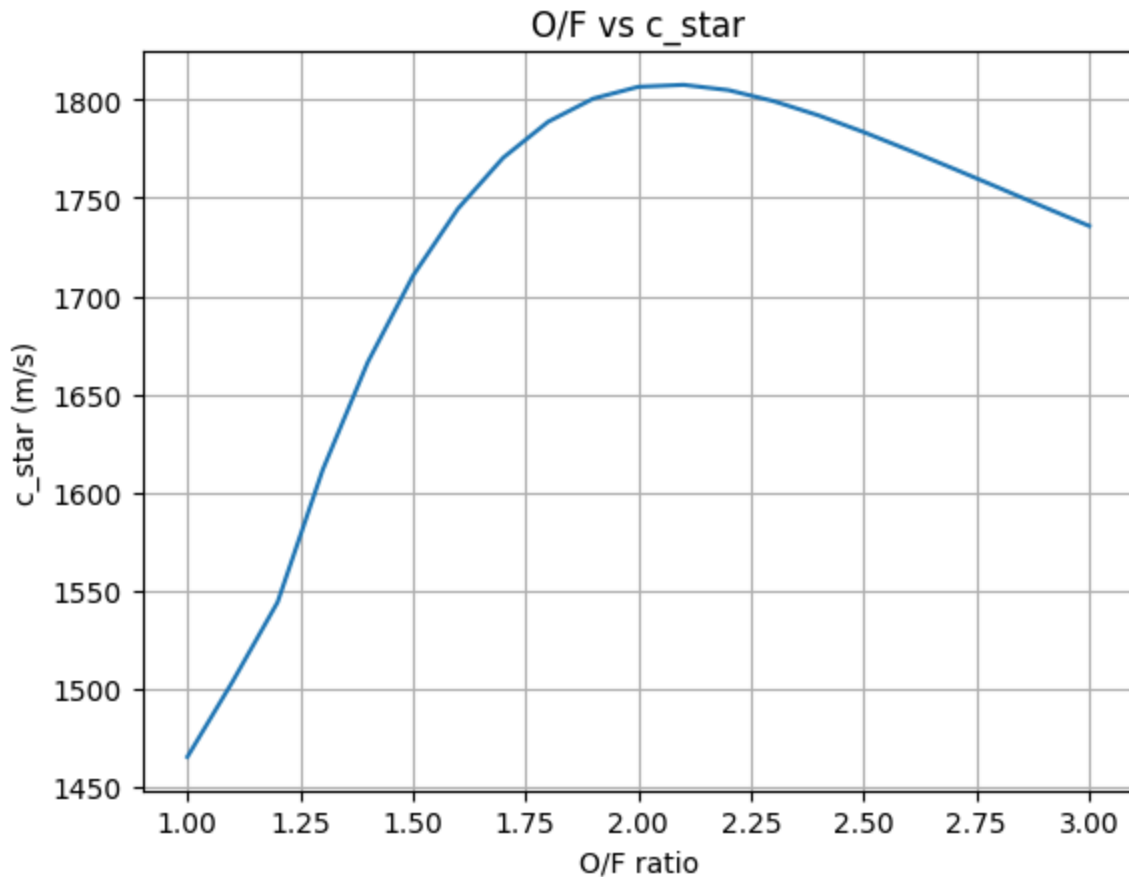
c_star_test = []

for line in lines:
    # ignore blank lines
    if not line.strip():
        continue

    words = line.split()
    if words[0] == 'CSTAR,':
        c_star_test.append(float(words[3]))

c_star_test = Q_(np.array(c_star_test), 'm/s')

plt.plot(mixture_ratios, c_star_test.magnitude)
plt.grid(True)
plt.xlabel('O/F ratio')
plt.ylabel('c_star (m/s)')
plt.title('O/F vs c_star')
plt.show()
```



```
In [37]: #Propellant 1: HTPB + LOX Looks like its best O/F ratio is 2.
r=2

Pe = Q_(14.7, 'psi') #From https://www.noaa.gov/jetstream/atmosphere/air-pressure
pressure_ratio = Pc_initial / Pe

fuel_density = Q_(902, 'kg/m^3') # https://www.jstage.jst.go.jp/article/rheology/40
oxidizer_density = Q_(1141, 'kg/m^3') #Value obtained from https://www.aqua-calc.co

gamma = 1.2643
c_star = Q_(1806, 'm/s')
Isp = Q_((2896.2+2784.9)/2, 'm/s') / g0 #Accounting for both equilibrium and frozen
area_ratio = get_area_ratio(pressure_ratio, gamma)
CF = get_thrust_coeff(pressure_ratio, gamma)
#Isp = CF*c_star / g0
```

```
In [38]: print('The Isp is', Isp.to('s'))
print('The c_star is', c_star.to('m/s'))
print('The area ratio is', area_ratio)
print('The thrust coefficient is', CF)
```

The Isp is 289.55657492354743 second
The c_star is 1806 meter / second
The area ratio is 7.856039134439211 dimensionless
The thrust coefficient is 1.564608789210233 dimensionless

```
In [39]: #For the masses of propellants, we are going to use Tsiokvsky Rocket equation
mo_mf = np.exp(delta_velocity / (Isp * g0))
```

```

#Using the propellant mass fraction of 0.9, we can use systems of equations to solve for
ms = mpl * (mo_mf - 1) / (10 - mo_mf)
mp = 9 * ms

fuel_mass = mp / (1 + r)
oxidizer_mass = fuel_mass * r

liftoff_mass = ms + mp + mpl

fuel_volume = fuel_mass / fuel_density

oxidizer_volume = oxidizer_mass / oxidizer_density

total_volume = fuel_volume + oxidizer_volume # not super important, as for hybrid w

mdot = mp / burn_time
#mdotf = fuel_mass / burn_time
#mdoto = oxidizer_mass / burn_time

Thrust_produced = mdot * Isp * g0

#Calculate what is the general thrust we are looking for
Thrust_requested = liftoff_mass * g0 * 2

```

```

In [54]: print('The starting mass to final mass ratio is', f'{mo_mf.magnitude:.3}')
print('The structural mass is' , f'{ms:.6}')
print('The propellant mass is' , f'{mp:.7}')
print('The total mass is' , f'{liftoff_mass}')
print('The mass of fuel is' , f'{fuel_mass}')
print('The mass of oxidizer is', f'{oxidizer_mass}')
print('The volume of propellant is' , f'{total_volume.to('m^3')}')
print('The volume of fuel is' , f'{fuel_volume.to('m^3')}')
print('The volume of oxidizer is', f'{oxidizer_volume}')
print('The average mdot is', f'{mdot:.5}')
#print('The mass flow rate of fuel is', f'{mdotf}')
#print('The mass flow rate of oxidizer is', f'{mdoto}')
print('The requested thrust is', f'{Thrust_requested.to('N')}')
print('The produced thrust is', f'{Thrust_produced.to('N')}')

```

```

The starting mass to final mass ratio is 2.25
The structural mass is 16087.7 kilogram
The propellant mass is 144789.1 kilogram
The total mass is 260876.74912265828 kilogram
The mass of fuel is 48263.02473679749 kilogram
The mass of oxidizer is 96526.04947359498 kilogram
The volume of propellant is 138.1044439660513 meter ** 3
The volume of fuel is 53.50667930908813 meter ** 3
The volume of oxidizer is 84.59776465696316 meter ** 3
The average mdot is 1809.9 kilogram / second
The requested thrust is 5118401.817786556 newton
The produced thrust is 5141007.55935413 newton

```

```

In [41]: #Nozzle Design

```

```

At = Thrust_produced / (CF*Pc_initial)
Rt = np.sqrt(At / np.pi)

```

```

Dt = 2 * Rt

Ae = At * area_ratio
Re = np.sqrt(Ae / np.pi)
De = 2 * Re

nozzle_length_ratio = 4

Ln = Dt * nozzle_length_ratio

```

```

In [51]: print('Throat Area', At.to('m^2'))
print('Throat Radius', Rt.to('m'))
print('Throat Diameter', Dt.to('m'))
print('Exit Area', Ae.to('m^2'))
print('Exit Radius', Re.to('m'))
print('Exit Diameter', De.to('m'))
print('Nozzle Length', Ln.to('m'))

```

```

Throat Area 0.4765664817970273 meter ** 2
Throat Radius 0.3894814791229761 meter
Throat Diameter 0.7789629582459522 meter
Exit Area 3.7439249311594587 meter ** 2
Exit Radius 1.0916630976258306 meter
Exit Diameter 2.183326195251661 meter
Nozzle Length 3.1158518329838087 meter

```

```

In [52]: #Calculating the initial mdot to find the initial mdoto and mdotf
mdot_initial = Pc_initial * At / c_star
print('The initial mdot is', mdot_initial.to('kg/s'))

mdotf_initial = mdot_initial / (r+1)
print('The initial fuel mass flow rate is', mdotf_initial)
mdoto_initial = mdotf_initial * r
print('The initial oxidizer mass flow rate is', mdoto_initial)

```

```

The initial mdot is 1819.3855072257147 kilogram / second
The initial fuel mass flow rate is 606.4618357419049 kilogram / second
The initial oxidizer mass flow rate is 1212.9236714838098 kilogram / second

```

```

In [53]: #Chamber size
Rf = Q_(1.5, 'm') / 2
#Rf = Df / 2
a = Q_(0.1, '(in/s) / (lb/in^2/s)**0.68')
n = 0.68

Ri = (Rf**(2*n+1) - (a * (2*n+1) * (mdoto_initial/np.pi)**n * burn_time))**(1/(2*n+1))
Di = 2*Ri
print('The inner diameter is', Di)

rdot_initial = a * (mdoto_initial.to('lb/s') / (np.pi * Ri.to('in')**2))**0.68
print('the rdot initial is', rdot_initial)
rdot_initial = Q_(rdot_initial.magnitude, 'in/s') #I'm not sure how its getting in*
print('the rdot initial is', rdot_initial)

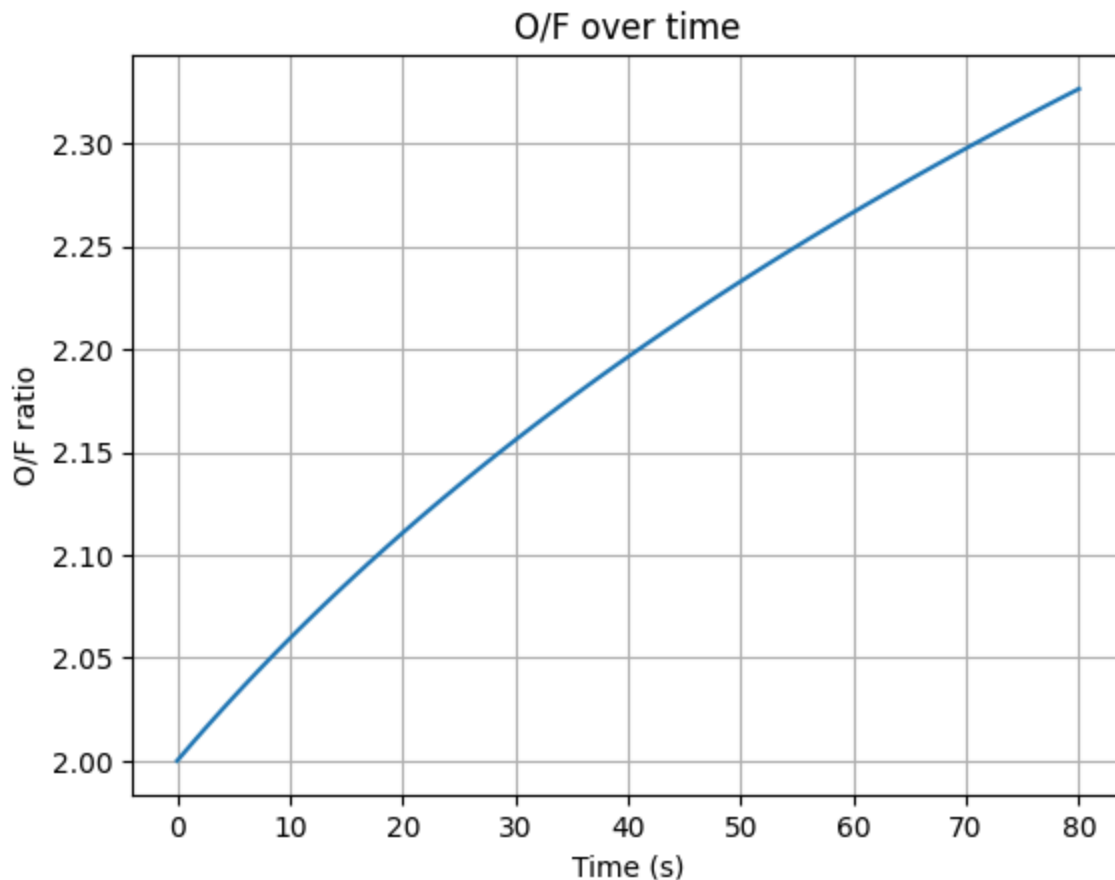
Lc = mdotf_initial / (2*np.pi * Ri * fuel_density * rdot_initial)
print('the chamber length is', Lc.to('m'))

```

The inner diameter is 0.9853795692407057 meter
the rdot initial is 0.17421516633465325 inch ** 1 / second
the rdot initial is 0.17421516633465325 inch / second
the chamber length is 49.082179118907696 meter

```
In [45]: timetest = Q_(np.linspace(0, burn_time, 100) , 's')
```

```
In [46]: O_F_ratio_progression = get_O_F_ratio(a, n, mdoto_initial.to('lb/s'), timetest.to('
O_F_ratio_progression = Q_(O_F_ratio_progression.magnitude, 'dimensionless') #I get
plt.plot(timetest, O_F_ratio_progression.magnitude)
plt.grid(True)
plt.title('O/F over time')
plt.xlabel('Time (s)')
plt.ylabel('O/F ratio')
plt.show()
```



```
In [47]: #taking our oxidizer mass amount, we can find the volume required.
tank_diameter = Q_(3, 'm')
tank_length = oxidizer_volume / (np.pi * (tank_diameter / 2)**2)
print('The oxidizer volume is', oxidizer_volume)
print('The length of tank to store oxidizer is', tank_length)
```

The oxidizer volume is 84.59776465696316 meter ** 3
The length of tank to store oxidizer is 11.968135484160468 meter

```
In [48]: from scipy.integrate import solve_ivp
#print(Isp, mp, burn_time, gamma)
sol = solve_ivp(
    vertical_launch, [0, burn_time.magnitude], [0, 0, liftoff_mass.to('kg').magnitu
```

```
args=(Isp.to('s').magnitude, mp.to('kg').magnitude, burn_time.magnitude, gamma)
dense_output=True
)
```

```
In [49]: time = np.linspace(0, burn_time.magnitude, 100)
z = sol.sol(time)

acceleration = np.gradient(z[0, :], time)

fig, axes = plt.subplots(3, 1)

axes[1].plot(time, z[0, :])
axes[1].set_ylabel('Velocity (m/s)')
axes[1].grid(True)

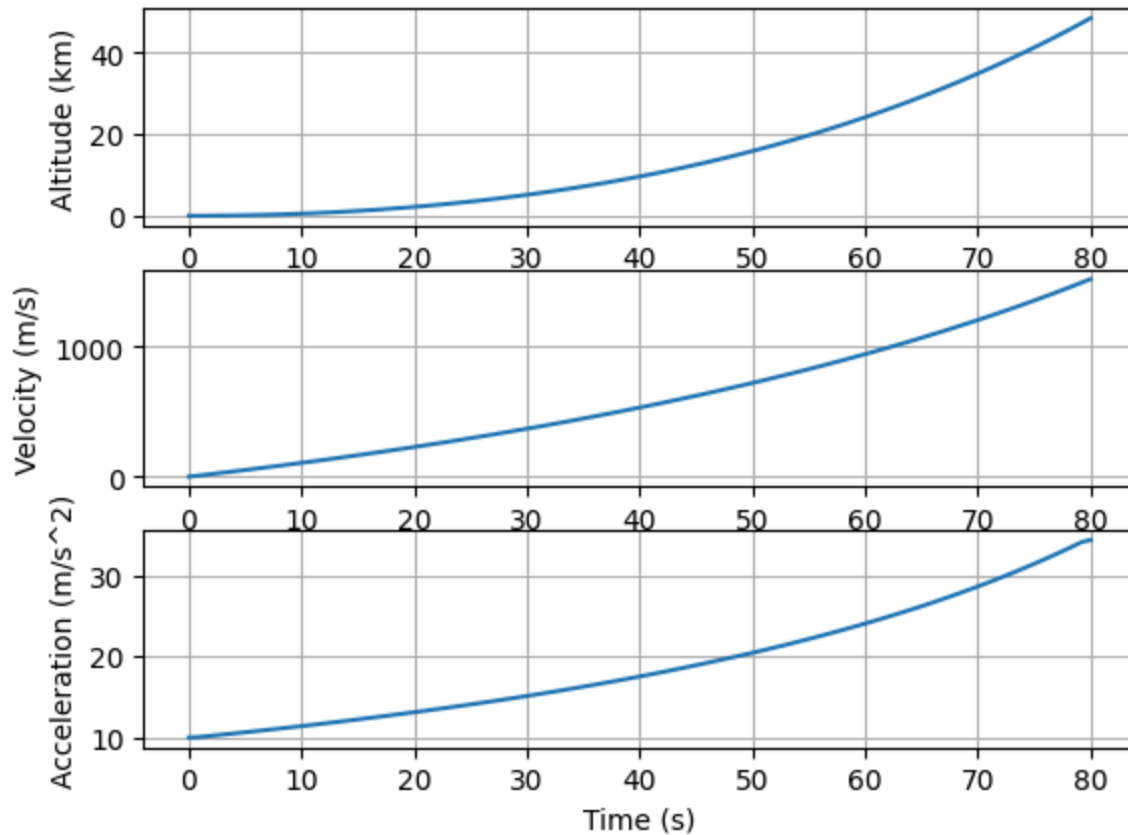
axes[0].plot(time, z[1, :] / 1000)
axes[0].set_ylabel('Altitude (km)')
axes[0].grid(True)

axes[2].plot(time, acceleration)
axes[2].set_xlabel('Time (s)')
axes[2].set_ylabel('Acceleration (m/s^2)')
axes[2].grid(True)

plt.suptitle('Rocket Performance for HTPB + LOX', fontsize=16, fontweight='bold')

plt.show()
#Remove drag component.
```

Rocket Performance for HTPB + LOX



```
In [50]: acceleration_initial = Q_(10, 'm/s^2') #From graphs, Looks like the acceleration ri
max_pressure_oxidizer = oxidizer_density*(g0+acceleration_initial)*tank_length
yield_strength = Q_(310, 'MPa') #https://asm.matweb.com/search/specificmaterial.asp
tensile_strength = Q_(276, 'MPa')
Tw = min((yield_strength)/1.1, (tensile_strength)/1.25) #Assuming uncrewed

t_c_oxidizer = max_pressure_oxidizer * (Df/2) / Tw

print('Max working stress', Tw)
print('Tank Thickness' ,t_c_oxidizer.to('mm'))
```

Max working stress 220.8 megapascal
 Tank Thickness 1.8377600520171926 millimeter

In []: